

SEDust: User Manual

Abstract

This manual describes the model-agnostic dust library under SEDust/sed/. It computes the infrared emission spectrum *and* the transport optics (extinction, scattering, asymmetry) of a dust mixture for an arbitrary local radiation field, and is intended to be linked into a Fortran 3D radiative-transfer (RT) code. It handles several dust models as peers — the HD23 *astrodust*+PAH model, the Draine & Li (2007) silicate+carbonaceous model, the Zubko, Dwek & Arendt (2004) BARE-GR-S model, and arbitrary file-defined models — through one object type, one emission call, and one extinction call. The numerically validated stochastic-heating solver is reused unchanged; the library is an additive layer on top of it.

This version is the **scalar** branch (v1.00): unpolarized cross sections and unpolarized emission. There is no polarized-transfer API here — no dichroic extinction matrix, no aligned-grain scattering matrix. Use v1.20 for those.

Contents

1. Overview	2
2. Building the library	2
3. Models and builders	3
4. Computing emission	4
4.1 Single-cell exact solve	4
4.2 Solver choice (m%stoch_method)	4
4.3 Equilibrium-temperature options	4
4.4 Precomputed table + interpolation	5
4.5 Optional error status	5
5. Computing extinction	6
6. The extreme-ultraviolet extension	7
6.1 Where the optics come from in the extended band	8
6.2 The shape approximation, and why it is bounded	8
6.3 What else the extension does, and does not, change	9
6.4 How far each model can be checked	9
7. Standalone programs	10
7.1 calc_kext.x — size-integrated transport optics	10
7.2 main_astrodust.x — the astrodust production SED	11
7.3 main_dl07.x — the DL07 reference SED	11
7.4 make_enthalpy.x — enthalpy tables (diagnostic)	11
8. The model object and accessors	11

9. The generic file loader	12
10. Linking into a 3D RT code	13
11. Units and conventions	13
12. Data files	13
12.1 Transport-optics products	14

1. Overview

A *dust model* is represented by a single derived type, `dust_model_t`. You build it once (per RT run), then call `dust_emission` once per RT cell with that cell's local mean intensity:

```
use dust_lib
type(dust_model_t) :: m
real(wp), allocatable :: J_lam(:), lamI_total(:)

call build_astrodust(m, qtab, sizedist, 200, 2.7_wp, 5.0e3_wp) ! once
allocate(J_lam(m%NLAM), lamI_total(m%NLAM))
do icell = 1, ncells
    ! ... assemble J_lam(:) (mean intensity on the grid m%lam) for this cell ...
    call dust_emission(m, J_lam, lamI_total) ! per cell
    ! ... lamI_total(:) is lambda*I_lambda / N_H for the cell ...
end do
```

Design. The library wraps the existing, FIR-validated solver core (`sed_grain_loop`, `calc_Teq`, `calc_P`) without modifying it. A model is a set of grain *populations* (one per species and charge state), each carrying its own size distribution, absorption cross sections, enthalpy, and Planck-integral tables. `dust_emission` loops the populations through the solver and sums their emission into named output *channels*.

One active model at a time. The module-global grids (`lam`, `T_first`, ...) hold the *currently built* model's working set. Calling `build_*` makes that model active. This is sufficient for an RT run that uses a single dust model; to switch models, rebuild.

2. Building the library

From `SEDust/sed/`:

```
make libsedust.a # the library archive (link this into your RT code)
```

The compiler is gfortran. The default flags are `-O3 -ffree-line-length-none -fopenmp` plus a warning set — deliberately *portable*, so that a `libsedust.a` built on one node runs on any other. A `-march=native` line is present but commented out; enabling it can change the last digits through FMA contraction. `libsedust.a` is a static archive of all library objects; the Fortran `.mod` files are written alongside it in `SEDust/sed/`. The Makefile declares `.NOTPARALLEL`, because every executable recipe compiles the shared module sources into the same directory and `make -j` would race on the `.mod` files.

3. Models and builders

Each builder fills a `dust_model_t` and defines the model’s output channels (Table 1).

builder	model	channels	notes
<code>build_astrodust</code>	HD23 astrodust+PAH	AD, PAH	$N_C=417$, D16 graphite
<code>build_dl07</code>	Draine & Li 2007	SIL, CARB	$N_C=470$, D03 graphite
<code>build_zubko</code>	Zubko (ZDA 2004) BARE-GR-S	PAH, GRA, SIL	neutral PAH only
<code>build_from_files</code>	file-defined	CH1, CH2, ...	generic loader

Table 1: Built-in model builders. Each sets the model’s own optics/abundance parameters internally.

Signatures (all optional arguments in brackets; every optional is a keyword argument in the order shown):

```
call build_astrodust(m, qtable_path, sizedist_path, NT, T_lo, T_hi &
                    [, status] [, lam_min] [, astrodust_index_path])
call build_dl07 (m, qtable_path, sizedist_path, sd_index, U, NT, T_lo, T_hi &
                [, status] [, lam_min])
call build_zubko (m, config_path, data_dir, NT, T_lo, T_hi [, status])
call build_from_files(m, descriptor_path, data_dir, NT, T_lo, T_hi [, status])
```

- `NT, T_lo, T_hi`: number and range [K] of the internal temperature grid (typical 200, 2.7, 5000).
- `sd_index` (DL07): WD01 size-distribution index — 7 = MW $R_V=3.1$, $q_{PAH}=4.58\%$; 29/30/31 = LMC2; 32 = SMC.
- `U` (DL07): ISRF intensity scaling (the DL07 reference uses $U=1$).
- `data_dir` (Zubko / files): directory holding the data files, e.g. `data/zubko/`.
- `status` (optional, all builders): error report; see §4.5. When present, a missing or malformed input file is reported through it and the model is not built, so an RT host can recover.
- `lam_min` (optional, astrodust and DL07): shortest wavelength [μm] the model must cover, for a host that transports ionizing radiation. See §6. **Omitting it reproduces the previous behavior exactly** — the grid, and every number derived from it, is unchanged.

- `astrodust_index_path` (optional, `astrodust` only): the dielectric function that supplies the optics below $0.0912\mu\text{m}$. It must be the file `qtable_path` was computed from — same porosity, iron fraction and axial ratio — or the model changes material at the seam. Omitted, it is the default that pairs with the shipped Q table:

```
data/dielectric/index_DH21Ad_P0.20_0.00_1.400 <->
tmatrix/output/q_astrodust_P0.20_Fe0.00_1.400.dat
```

Two routes to the Zubko model. `build_zubko` evaluates the ZDA log-polynomial size-distribution *formula* (coefficients read from `ZDA_BARE_GR_S_Config.dat`) on the optics-table radii; `build_from_files` with `data/zubko/zubko_descriptor.txt` instead interpolates the *tabulated* `SzDist` files onto the same radii. The two shapes agree to $\sim 10^{-5}$, but the distributed tables carry a normalization that sits below $Ag(a)$ of the config by 0.79% (PAH), 1.13% (graphite) and 0.41% (silicate), and they are sampled at only 24 / 62 / 63 radii, so interpolation near a_{max} adds a little more. Together this moves the size-integrated C_{ext} by up to 2.10% (median 1.02%). **The formula is the default and is faithful to the ZDA definition;** the tabulated route exists to reproduce the benchmark’s own size grid.

4. Computing emission

4.1 Single-cell exact solve

```
call dust_emission(m, J_lam, lamI_total [, lamI_chan] [, status])
```

- `J_lam(m%NLAM)`: the local *mean intensity* on the grid `m%lam` [μm]. For a scaled Mathis ISRF use `call J_Mathis(U, m%lam, J_lam)` from module `radfield`.
- `lamI_total(m%NLAM)`: the summed SED, $\lambda I_{\lambda}/N_{\text{H}}$ [$\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1} \text{H}^{-1}$] — the HD23 convention.
- `lamI_chan(m%NLAM, m%n_channel)` (optional): the SED of each channel.
- `status` (optional): error report; see §4.5.

4.2 Solver choice (m%stoch_method)

The field `m%stoch_method` selects how each grain is heated (Table 2); `dust_emission` honors it.

4.3 Equilibrium-temperature options

Two options replace the full stochastic solve when an equilibrium approximation is wanted:

(1) Equilibrium temperature for each grain type and size. Set `m%stoch_method = 'equil'` and call `dust_emission`. Every grain is placed at its own equilibrium temperature T_{eq} , computed from *that grain’s* absorption cross section, and emits $C_{\text{abs}} B_{\lambda}(T_{\text{eq}})$. No stochastic excursions are computed.

m%stoch_method	method	note
'heuristic'	GD look-ahead T-window narrowing	default ; fastest, serial
'draine'	Guhathakurta & Draine (1989)	reference GD
'qm'	energy-space transition matrix (thermal-discrete)	most detailed
'equil'	single-grain equilibrium (no stochastic)	see §4.3

Table 2: Stochastic-heating solver options. The first three resolve the grain temperature *distribution* (PAH/NIR features); 'equil' forces equilibrium.

(2) A single equilibrium temperature for the whole model.

```
call dust_emission_single_teq(m, J_lam, lamI_total [, Teq_out])
```

All dust shares one temperature, found from the type- and size-integrated (dn-weighted) total absorption cross section $C_{\text{abs}}^{\text{tot}}(\lambda) = \sum_{\text{pop}} \sum_a \text{dn}(a) C_{\text{abs}}(\lambda, a)$. The single T_{eq} satisfies $\int C_{\text{abs}}^{\text{tot}} J d\lambda = \int C_{\text{abs}}^{\text{tot}} B(T_{\text{eq}}) d\lambda$, and the emission is $C_{\text{abs}}^{\text{tot}} B_{\lambda}(T_{\text{eq}})$. The optional `Teq_out` returns that temperature.

Both options conserve energy (the emitted bolometric power equals the absorbed power); they differ from the stochastic solve only in the SED *shape* — a smooth modified blackbody rather than one carrying PAH/NIR features.

4.4 Precomputed table + interpolation

When the field in each cell is a fixed spectral *shape* scaled by an intensity U , precompute a table once and interpolate per cell:

```
type(dust_emis_table_t) :: tab
call dust_build_table(m, J_ref, U_grid, tab [, status]) ! once
call dust_emission_interp(tab, U, lamI_total [, lamI_chan] [, status]) ! per cell
```

$J_{\text{ref}}(m\%NLAM)$ is the reference field shape (at $U=1$) and $U_{\text{grid}}(:)$ an ascending intensity grid. The table is built from exact solves, so it is exact at the grid points; interpolation is log–log in U . The stored emissivities are floored at 10^{-300} before the log, so a grid node whose true value is 0 comes back as 10^{-300} rather than exactly 0. Both calls take an optional final status (§4.5).

Caveat. Stochastic emission is a *nonlinear* functional of the full $J(\lambda)$, not of a scalar. The table is valid only when the cell field is $U \times (\text{fixed } J_{\text{ref}})$. For cells whose field *shape* departs from J_{ref} (e.g. hardened spectra near hot stars), use the cell-by-cell exact `dust_emission`.

4.5 Optional error status

The builders and the emission/table calls each accept an optional final status argument (`integer, intent(out)`). When it is present, an invalid model, argument, or input file is reported through it (`status = 0` on success, nonzero on failure) and the process keeps running, so a 3D RT host can recover; when it is omitted, the same condition prints

a message and stops the run, which the CLI drivers rely on. For the builders the nonzero value only names the failing stage: the contract is simply 0 = built, nonzero = build failed.

- `dust_emission`: 1 unknown `stoch_method`; 2 'qm' selected but a population's radii are unset.
- `dust_build_table`: 1 `size(J_ref) ≠ m%NLAM`; 2 `size(U_grid) < 2`; 3 `U_grid` not positive and strictly increasing.
- `dust_emission_interp`: 1 $U \leq 0$; 2 `size(lamI_total) ≠ tab%NLAM`; 3 `lamI_chan` present but not (`tab%NLAM`, `tab%n_channel`).
- `build_astrodust`: 1 Q -table load failed; 2 size-distribution load failed; 3 the astro dust dielectric function failed to load (raised only when `lam_min` asks for the EUV band); 4 `lam_min` is shorter than that dielectric function's own shortest wavelength ($1.00003 \times 10^{-4} \mu\text{m}$), which is refused rather than served with a refractive index frozen at the table boundary.
- `build_dl07`: 1 Q -table load failed; 2 size-distribution load failed; 4 `lam_min` is shorter than the D03 dielectric functions ($6.1992 \times 10^{-5} \mu\text{m}$ — the longer-reaching of astrosilicate and graphite), refused for the same reason. (There is no code 3 on this path: this model needs no separate EUV dielectric function, because its optics are D03 Mie at every wavelength.)
- `build_zubko`: 1 config read; 2 fewer than 3 components; 3 a component's optics read; 4 grid inconsistent; 5 a component's calorimetry read failed.
- `build_from_files`: 1 descriptor open; 2 too many pop: lines; 3 invalid channel; 4 no pop: lines; 5 optics read; 6 grid inconsistent; 7 size-distribution read; 8 calorimetry read failed.

5. Computing extinction

`dust_extinction` is the extinction counterpart of `dust_emission`: a transfer code takes its opacity from the same model object, and on the same wavelength grid `m%lam`, as its emission.

```
call dust_extinction(m, Cext, Cabs, Csca [, gbar] [, albedo] [, status])
```

- `Cext`, `Cabs`, `Csca` (`m%NLAM`): extinction, absorption and scattering cross sections per H atom [cm^2/H], with $C_{\text{ext}} = C_{\text{abs}} + C_{\text{sca}}$.
- `gbar` (`m%NLAM`) (optional): the scattering-weighted asymmetry $\langle \cos \theta \rangle$, and 0 where nothing scatters.
- `albedo` (`m%NLAM`) (optional): $C_{\text{sca}}/C_{\text{ext}}$, and 0 where C_{ext} underflows to zero. It is derived here rather than by the caller so that every host gets the same convention at those wavelengths.
- `status` (optional): error report; see §4.5. It is 0 on success and 1 if an output array is not of size `m%NLAM`.

Each output is the plain binned sum over every population of the model, since $dn(a)$ already carries the bin width:

$$C_{\text{abs}}(\lambda) = \sum_{\text{pop}} \sum_a dn(a) C_{\text{abs}}(\lambda, a),$$

$$C_{\text{sca}}(\lambda) = \sum_{\text{pop}} \sum_a dn(a) C_{\text{sca}}(\lambda, a),$$

$$\langle \cos \theta \rangle = \frac{\sum dn C_{\text{sca}} g}{\sum dn C_{\text{sca}}}.$$

The values reproduce data/kext_astrodust_MW.dat to the precision with which that file is written, because the file is written by this very call (§7.): `calc_kext.x` builds the model and calls `dust_extinction`, so the tables under data/ are the optics an RT host gets in memory. The difference is only that the library returns the result on `m%lam` directly, so a host using its own wavelength grid need not interpolate off the grid the driver happened to write.

Every model scatters. All four builders now populate the extinction-side scattering optics, so `albedo` and `gbar` are physical for every model:

- **astrodust:** C_{sca} and $\langle \cos \theta \rangle$ from the random-orientation T-matrix Q table (and, below $0.0912 \mu\text{m}$, from Mie on the same dielectric function).
- **DL07:** Mie on the D03 astrosilicate and graphite dielectric functions (`q_silicate_full` / `q_graphite_full`), on the same random-orientation average ($\frac{1}{3} E \parallel c + \frac{2}{3} E \perp c$) as the absorption. The two carbonaceous charge states share one scattering description: charge shifts PAH absorption features, not the graphite sphere's scattering.
- **Zubko / file-defined:** the Q_{sca} and g columns of the model's own DustEM tables, i.e. the authors' own multilayer-sphere solution, read from the same file as Q_{abs} .

A population that genuinely does not scatter leaves its scattering optics unallocated and contributes zero. In the `astrodust` model the PAHs are exactly that case — PAH scattering is Rayleigh-negligible — so they enter extinction through absorption only, and the `astrodust` grains carry all the scattering and the asymmetry.

6. The extreme-ultraviolet extension

The `astrodust` model's wavelength grid is the T-matrix Q table's, which stops at $0.0912 \mu\text{m}$ — 13.6 eV, the Lyman limit. A host that transports ionizing radiation needs shorter wavelengths, so both `build_astrodust` and `build_dl07` take an optional `lam_min` [μm]:

```
call build_astrodust(m, qtab, sizedist, NT, T_lo, T_hi, lam_min=1.001e-4_wp)
```

Log-spaced points are then prepended from `lam_min` up to the table, at a step no coarser than the table's own $\Delta \ln \lambda = 0.01156$ at its short-wavelength end, with `m%lam(1)` set to `lam_min` exactly. **Omitting `lam_min` prepends nothing and leaves the model bit-identical to before.**

6.1 Where the optics come from in the extended band

- **Astrodust grains.** Bohren–Huffman Mie for the *volume-equivalent sphere* on the Draine & Hensley (2021) dielectric function — the same material as the Q table, but not the same shape. See §6.2.
- **PAH / carbonaceous.** Above 21.4 eV ($1/\lambda = 17.25 \mu\text{m}^{-1}$) the DL07 PAH cross section is zero by construction, so graphite is the *whole* carbonaceous absorption there. Below that energy the D16 turbostratic-graphite sphere table is used as before; above it the code switches to Mie on the D03 graphite dielectric functions (random orientation, $\frac{1}{3}$ parallel + $\frac{2}{3}$ perpendicular), because the D16 table’s radius axis bottoms out at $10^{-3} \mu\text{m}$ while the astrodust size grid starts at $3.55 \times 10^{-4} \mu\text{m}$. In the Rayleigh limit $Q_{\text{abs}} \propto a$, so clamping a smaller grain to the table’s first radius overestimates its absorption by $10^{-3} \mu\text{m}/a$ — up to $2.4\times$ for the smallest PAH the size distribution populates ($4.22 \times 10^{-4} \mu\text{m}$). *This branch is unreachable on the unextended grid:* its shortest wavelength gives $1/\lambda = 10.96 < 17.25$, so every previous result is unaffected.
- **DL07 model.** Its optics are Mie on the D03 dielectric functions at *every* wavelength, so nothing changes character across the band and the extension is a grid extension only.
- **Zubko model.** No extension, and none possible: its tabulated optics already span 10^{-3} to $10^4 \mu\text{m}$, and those tables *are* the model definition, so there is no dielectric function to extend them with.

6.2 The shape approximation, and why it is bounded

The Q table is a random-orientation T-matrix solution for an oblate spheroid of axial ratio $b/a = 1.4$; the extended band uses the volume-equivalent *sphere*. The justification is **not** the anomalous-diffraction argument $|m - 1| \ll 1$: for this material $|m - 1| = 0.77$ at the seam and still 0.56 at 20 eV. It rests instead on the two limits that actually apply:

- **Small grains are in the Rayleigh limit** ($x = 2\pi a/\lambda = 0.033$ at the seam for the smallest populated radius, and only 0.24 at 100 eV). There the ellipsoid polarizability is $\alpha_j = V(\epsilon - 1)/\{4\pi[1 + L_j(\epsilon - 1)]\}$, so shape enters *only* through $L_j(\epsilon - 1)$. And $|\epsilon - 1|$ falls from 1.86 at 13.5 eV to 1.10 at 20 eV, 0.28 at 40 eV and 0.061 at 100 eV: the depolarization factors drop out and sphere and spheroid converge on the same C_{abs} .
- **Large grains are in the geometric-optics limit**, where $C_{\text{ext}} \rightarrow 2 \times (\text{orientation-averaged projected area})$. By Cauchy’s theorem that average is $S/4$, and for $b/a = 1.4$ the spheroid’s $S/4$ exceeds πa_{eff}^2 of its volume-equivalent sphere by 2.12%. Mie is therefore low by $1 - 1/1.0212 = 2.08\%$ in that limit. **The error converges to a constant, not to zero**, however far into the EUV one goes.

Consequently the error does *not* fall monotonically with photon energy. Only $a \lesssim 2 \times 10^{-3} \mu\text{m}$ improves monotonically; $a \gtrsim 0.01 \mu\text{m}$ gets *worse* between 13.6 and 30 eV before recovering, and the largest sizes settle at a nearly energy-independent -1.9 to -2.0% . Over the whole band and the whole size range the error stays within about 2%; `sed/src/q_astrodust.f90` tabulates it size by size.

Seam continuity. In the size-integrated curve the step across $0.0912 \mu\text{m}$ is -1.54% in C_{ext} and -1.90% in C_{abs} — *smaller* than the neighboring steps of the table’s own grid (-1.95% and -1.72% for C_{ext} ; -2.20% and -1.94% for C_{abs}),

so absorption and extinction join without a visible discontinuity. Scattering does show one: C_{sca} steps by +0.74% where its neighbors step by −0.39% and −0.36%, and the albedo by +2.31% where its neighbors give +1.59% and +1.39% — a seam of roughly one percentage point, consistent with the 2% shape error being carried mostly by Q_{sca} .

6.3 What else the extension does, and does not, change

- **The hardest photon the solver assumes** is no longer the fixed 13.6 eV Lyman limit but $\max(13.6 \text{ eV}, hc/\lambda_1)$, taken from the grid itself. On the unextended grid $hc/0.0912 \mu\text{m} = 13.595 \text{ eV} < 13.6 \text{ eV}$, so the value there is unchanged.
- **The Planck integrals are anchored to the table range.** The internal log- λ integration grid for $\kappa_B(T, a)$ keeps its step and its sample points over the optics-table range and merely *prepends* points below it, so widening the model grid cannot coarsen the sampling of the range that carries the integrand. Measured: with a $1.001 \times 10^{-4} \mu\text{m}$ floor, κ_B is unchanged to round-off over the thermal range ($\max |\Delta\kappa_B|/\kappa_B = 3.4 \times 10^{-16}$ for $T \leq 3000 \text{ K}$, and exactly zero below $\sim 2400 \text{ K}$); κ_{CMB} is bit-identical. The only nonzero difference is 1.1×10^{-8} at the very top of a 5000 K grid, where the Wien tail genuinely reaches into the extended band — that is real extra emission, not a sampling artifact. Without the anchoring, spending a fixed budget of points on the widened interval would have moved κ_B by $\sim 10^{-4}$ for a purely numerical reason.
- **Stochastic heating by hard photons is not solved by this change.** The extension fixes the photon-energy mapping and the absorbed-energy accounting. It does not address the finite enthalpy grid: upward transitions above the top of that grid are accumulated into the highest temperature bin, and for the smallest PAHs a 54 eV photon already exceeds $H(5000 \text{ K})$. Nor does it add photoionization, electron energy loss, fragmentation or PAH destruction. Treat the extended band as extinction and absorbed-energy physics, not as a validated hard-EUV PAH *emission* model.

6.4 How far each model can be checked

author-provided data	coverage	what it contains
Draine, kext_albedo_WD_MW_3.1_60_D03 (DL07 / WD01)	10^{-4} – $10^4 \mu\text{m}$ $= 1.24 \times 10^{-4} \text{ eV}$ – 12.4 keV	λ , albedo, $\langle \cos \theta \rangle$, C_{ext}/H , K_{abs} , $\langle \cos^2 \theta \rangle$
HD23 astrodust release (extinction.dat, scattering.dat, wav.dat)	0.1 – $3 \times 10^4 \mu\text{m}$ $= 4.1 \times 10^{-5}$ – 12.4 eV	absorption and scattering τ/N_{H} only. No $\langle \cos \theta \rangle$ product exists anywhere in the release.
Zubko ZDA optics tables	10^{-3} – $10^4 \mu\text{m}$ $= 1.24 \times 10^{-4} \text{ eV}$ – 1.24 keV	Q_{abs} , Q_{sca} , Q_{ext} , g for each radius

Table 3: What the model authors published, and how far it reaches.

The consequence is blunt: **the astrodust EUV band has no external reference at all**, because the HD23 release stops

at 12.4 eV — exactly where the extension begins. Its accuracy claim rests entirely on the internal argument of §6.2 and on the seam being continuous.

The DL07 model *can* be checked, because Draine published the same model over the whole range. Comparing data/kext_dl07_MW_euv.dat with kext_albedo_WD_MW_3.1_60_D03.all_2003 on the reference’s own wavelengths (and counting only points where the reference grid is no finer than ours — elsewhere the comparison measures our grid, not our optics) gives, per decade in λ , mean $|\Delta C_{\text{ext}}|/C_{\text{ext}}$ of 0.056–0.86%, mean $|\Delta \text{albedo}| \leq 0.0019$ and mean $|\Delta \langle \cos \theta \rangle| \leq 0.00054$. calc_kext.x dl07 euv prints that table. The two residual features are worth naming: the excluded points cluster at the X-ray absorption edges (Fe K, Si K, Mg K, Fe L, O K, C K), which the reference brackets far more finely than our $\Delta \ln \lambda = 0.0116$ log grid can resolve; and the largest single deviation, 7.9% in the 1–10 μm decade, is confined to the 6.2, 7.6 and 7.9 μm PAH C–C features, where Draine’s 2003 table uses a different vintage of the PAH cross sections.

7. Standalone programs

make in SEDust/sed/ builds four executables. All of them are run *from* SEDust/sed/, because every data path is relative to ../.

7.1 calc_kext.x — size-integrated transport optics

Builds a model and calls dust_extinction on it, then writes the result under ../data/. Every model goes through the same two calls, so the files it writes *are* the optics an RT host gets in memory.

```
./calc_kext.x astrodust [euv | <lam_min in um>]
./calc_kext.x dl07 [euv | <lam_min in um>]
./calc_kext.x zubko [formula | table]
./calc_kext.x from_files <descriptor> [data_dir]
```

With no argument it prints its usage and stops. euv carries the grid down to whatever the model’s own dielectric function covers; an explicit number requests a specific floor in μm . zubko defaults to formula. from_files takes the descriptor path, and the data directory defaults to the descriptor’s own directory.

Columns are

$\lambda[\mu\text{m}]$ albedo $\langle \cos \theta \rangle$ C_{ext}/H C_{abs}/H C_{sca}/H

in μm and cm^2 per H nucleon, with a trailing K_{abs} [cm^2/g] column when the model supplies its dust mass per H. The first four columns are in the order of Draine’s kext_albedo tables, so a reader written for those files (for instance the par%ion_dust_kext reader of the MoCHII host, which takes four reals from each row and uses λ , albedo and C_{ext}) parses these unchanged. Note the fifth column differs from Draine’s: his is K_{abs} [cm^2/g], ours is C_{abs}/H [cm^2/H], so no dust-mass normalization is folded into our numbers.

After writing, the program checks $C_{\text{ext}} = C_{\text{abs}} + C_{\text{sca}}$ and the ranges of albedo and $\langle \cos \theta \rangle$, and — for astrodust and DL07 — prints the comparison against the author-provided data of Table 3.

These files are transport optics, not a dust model. They carry no size-resolved $C_{\text{abs}}(\lambda, a)$, no grain enthalpy and no Planck integral, so stochastic heating and thermal emission cannot be computed from them. A host that needs emission must build the model and take the same curves from `dust_extinction`, which is what keeps the absorbed power and the re-emitted spectrum referring to one and the same grain.

7.2 main_astrodust.x — the astro dust production SED

Solves the astro dust+PAH SED for a single Mathis-ISRF cell at $U = 1.585$ ($\log U = 0.20$, the HD23 best fit), on a 200-point $\log T$ grid from 2.7 to 5000 K. Reads the T-matrix Q table `../tmatrix/output/q_astro dust_P0.20_Fe0.00_1.400.dat` and `../data/release/size_distribution.dat`; writes `output/astro dust_irem_ours_<tag>S1.dat`, `... S2.dat` and `... PAH.dat`, each with λ [μm] and $\lambda I_{\lambda}/N_{\text{H}}$. Optional arguments, in any order, each of which tags the output filename so a non-default run cannot clobber the production one: `qm`, `qm_dbcon`, `draine` (solver); `logU=X`; `nstate=N`, `nisrf=N` (QM solver sizes); `induced` (turn the stimulated-emission factor on); `photcut`; `c2` (Stage-1 density-corrected prefactor); `mathis_orig`.

7.3 main_dl07.x — the DL07 reference SED

Solves the Draine & Li (2007) silicate + carbonaceous SED at $U = 1$ with the WD01 size distribution. `./main_dl07.x [model] [qm|draine]`, where `model` is one of `mw31_00` `... mw31_60` (default `mw31_60`), `lmc2_00`, `lmc2_05`, `lmc2_10`, `smc`; an unknown name is rejected with the valid list. Writes `output/dl07_sed_ours_<model>[_qm|_draine].dat` with columns λ , total, silicate and carbonaceous $\lambda I_{\lambda}/N_{\text{H}}$.

7.4 make_enthalpy.x — enthalpy tables (diagnostic)

Tabulates $U(T, a_{\text{eff}})$ for the two astro dust enthalpy stages and writes `output/enthalpy_S1.dat` and `output/enthalpy_S2.dat` (201 log-spaced temperatures from 2.7 to 5000 K $\times$ the 169 radii of `../data/dielectric/DH21_aeff`, T outer and a inner). It takes no arguments. **It is not required to compute an SED:** the solver calls the closed-form Debye sums directly in its inner loop. Use it to compare the two stages, to plot $U(T)$, or to spot-check against published DL01 values.

8. The model object and accessors

```
type :: dust_model_t
  character(len=32) :: name
  integer :: NA, NLAM, NT
  real(wp), allocatable :: lam(:) ! (NLAM) [um]
  real(wp), allocatable :: aeff(:) ! (NA) [um]
  real(wp), allocatable :: T_first(:) ! (NT) [K]
  real(wp), allocatable :: log_T_first(:) ! (NT)
  type(grain_pop_t), allocatable :: pops(:)
```

```

integer :: n_channel
character(len=16), allocatable :: channel_name(:)
logical :: use_induced_emission ! default .false.
character(len=16) :: stoch_method ! default 'heuristic'
logical :: verbose ! default .false.
end type

```

Each `grain_pop_t` in `m%pops` carries its own `aeff`, `dn`, `Cabs`, `Csca`, `gsca`, `kappB`, `H` and `kappCMB` (plus cached logs). A model whose components have different size grids — Zubko is the case in the tree — is represented by populations of different lengths, which is why the solver takes the size count from `size(pops(ip)%dn)` rather than from `m%NA`.

Diagnostics. `m%verbose` (default `.false.`) keeps the library solve path silent; set it `.true.` to print the same solver diagnostics as the CLI drivers.

Convenience accessors (all in `dust_lib`):

```

n = dust_nlam(m) ! number of wavelengths
nc = dust_n_channel(m) ! number of output channels
lam = dust_lambda(m) ! allocatable copy of the wavelength grid [um]
nm = dust_channel_name(m, ic) ! name of channel ic

```

9. The generic file loader

`build_from_files` reads a small text *descriptor* and assembles a model from data files, with no code changes. One `pop`: line per population:

```

# data/zubko/zubko_descriptor.txt
name = zubko_bare_gr_s
# pop: <grain_type> <channel> <optics> <dnda_table> <calorimetry> <rho(0=use file)>
pop: pah 1 PAH_28_1201_neu.dat ZDA_BARE_GR_S_SzDist_PAH.dat Graphitic_Calorimetry_1000.dat 0.0
pop: gra 2 Gra_121_1201.dat ZDA_BARE_GR_S_SzDist_Gra.dat Graphitic_Calorimetry_1000.dat 0.0
pop: sil 3 suvSil_121_1201.dat ZDA_BARE_GR_S_SzDist_Sil.dat Silicate_Calorimetry_1000.dat 0.0

```

- `grain_type`: 'sil', 'pah', or 'gra' (selects the heat-capacity/mode model used by the 'qm' solver; ignored by the GD solvers, which use the supplied enthalpy directly).
- `channel`: integer output channel (1...); populations sharing a channel are summed.
- `optics`: a DustEM/Zubko Q -table (Q_{abs} per radius and wavelength); $C_{\text{abs}} = Q_{\text{abs}} \pi a^2$.
- `dnda_table`: a 2-column table a [μm] and dn/da [$\text{cm}^{-1} \text{H}^{-1}$].
- `calorimetry`: a specific-enthalpy table $u(T)$ [erg/g]; $H(T, a) = u(T) \rho \frac{4\pi}{3} a^3$.

- rho: grain density [g cm^{-3}]; 0.0 uses the value in the optics-file header.

All files are sought under `data_dir`. The coded builders (`build_astrodust/dl07/zubko`) are the recommended path for the built-in models; `build_from_files` is for adding new data-defined models.

10. Linking into a 3D RT code

Compile your RT source against `libsedust.a` with the `.mod` search path pointing at `SEDust/sed/`:

```
gfortran -I<sed> my_rt.f90 <sed>/libsedust.a -fopenmp -o my_rt.x
```

A complete minimal example is provided in `SEDust/sed/rt_example/use_dustlib.f90`: it builds a model, computes emission for one field, and prints the SED peak. No `ISO_C_BINDING` is needed — the RT code simply uses the Fortran module `dust_lib`.

Threading. `m` is read-only during `dust_emission`; all scratch is local. The solver uses OpenMP internally over grains. If your RT parallelizes over cells, take care to avoid oversubscription (nested parallelism); the default 'heuristic' solver is serial per call, which suits an outer cell-level parallel loop.

11. Units and conventions

- Wavelengths λ are in μm throughout; cross sections in cm^2 .
- J_λ is a mean intensity with the same units as the Planck function B_λ (specific intensity per steradian). The library's output $\lambda I_\lambda / N_H$ is then in $\text{erg s}^{-1} \text{cm}^{-2} \text{sr}^{-1} \text{H}^{-1}$.
- The CMB (2.725 K) is included in the heating field by `J_Mathis`.
- The induced-/stimulated-emission factor is off by default (the output is *net* emission, matching the HD23 and DL07spec released SEDs).

12. Data files

Model data lives under `SEDust/data/`:

- `astrodust / DL07`: the T-matrix Q -table (`../tmatrix/output/q_astrodust_*.dat`) and the HD23 release size distribution (`../data/release/size_distribution.dat`).
- `dielectric functions`: `../data/dielectric/` — the DH21 `astrodust` index `index_DH21Ad_P0.20_0.00_1.400` (which supplies the `astrodust` EUV band), the D03 `astrosilicate` and `graphite` indices, and the D16 `turbostratic-graphite` sphere table.
- `Zubko`: `../data/zubko/` — the ZDA `config/formula`, the tabulated size distributions, the DustEM Q -tables, and the calorimetry tables from the Camps et al. 2015 benchmark; a ready descriptor is `../data/zubko/zubko_descriptor.txt`.

12.1 Transport-optics products

`calc_kext.x` (§7.) writes these into SEDust/data/. The last column is the point that matters when choosing one: **nothing here is a free choice — each floor is fixed by the data the model is made of**, and a shorter floor is refused rather than served with a refractive index frozen at its table boundary.

file	rows	λ range [μm]	what sets the short-wavelength floor
<code>kext_astrodust_MW.dat</code>	1129	0.0912–39810	the T-matrix Q -table grid (no extension requested)
<code>kext_astrodust_MW_euv.dat</code>	1719	1.001×10^{-4} –39810	the DH21 astrodust dielectric function, whose own table stops at 1.00003×10^{-4}
<code>kext_dl07_MW_euv.dat</code>	1761	6.205×10^{-5} –39810	the D03 dielectric functions; the shorter-reaching of astrosilicate and graphite stops at 6.1992×10^{-5}
<code>kext_zubko_BARE_GR_S.dat</code>	1201	10^{-3} – 10^4	the ZDA optics table itself; this model has no extension and needs none

Table 4: Size-integrated transport optics written by `calc_kext.x`. All carry the columns of §7.. `kext_astrodust_MW.dat` is a tracked regression reference and keeps its original narrower field widths; it is reproduced byte for byte by `./calc_kext.x astrodust`.